

第24讲：一个神经元的 Backpropagation

Weight、Bias、Input 的梯度从头到尾怎么算

核心概念：【必须真正掌握】

本讲把“节点级 backward”组装成一个真正的神经元反向传播。目标不是背公式，而是理解每个 Weight、Bias、Input 为什么会得到对应的梯度，以及这些梯度的 Shape 为什么与原变量一致。



建议学习方式：先跟着一个 3 输入神经元手算 -> 再看向量公式 -> 最后运行 Gradient Check 与 PyTorch Autograd 对照。

一、本讲学习目标

从一个向量输入神经元的 Forward 开始，完整算出 z 、ReLU 输出、Error 和 Loss。

逐步推导 dL/dw_i 、 dL/db 、 dL/dx_i ，而不是直接背最终公式。

理解为什么 dL/dw 与 w Shape 一样， dL/dx 与 x Shape 一样。

理解同一个上游梯度 dL/dz 为什么会被不同输入 x_i 缩放，形成不同 Weight 梯度。

区分参数梯度、输入梯度和中间变量梯度。

用数值差分检查每一个 Weight、Bias、Input 梯度。

用 PyTorch Autograd 做三重对照，但不要求现在记 PyTorch 语法。

为第25讲两层 MLP Backpropagation 做准备。

核心概念：与第23讲的关系

第23讲把 backward 拆成 Add、Multiply、ReLU、Square 等节点规则；第24讲把这些节点真正装进一个“有多个输入、多个 Weight”的人工神经元里。

二、【原理】一个神经元的 Forward 是什么？

我们从你第7讲见过的一个神经元开始。输入向量：

$$\mathbf{x} = [x_1, x_2, x_3]$$

Weight 向量：

$$\mathbf{w} = [w_1, w_2, w_3]$$

神经元先做加权求和：

$$z = x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + b$$

也可以写成点积：

$$z = \mathbf{x} \cdot \mathbf{w} + b$$

然后经过激活函数：

$$\mathbf{a} = \text{ReLU}(z)$$

最后与 target 比较，本讲用最简单的平方误差：

$$L = (\mathbf{a} - y)^2$$

核心概念：输入 / 输出

输入是 $\mathbf{x}$ 、 $\mathbf{w}$ 、 b 、target；Forward 输出一个标量 Loss。Backward 的任务是回答：如果稍微改变 $\mathbf{w}$ 、 b 或 $\mathbf{x}$ ，Loss 会怎样变化？

三、【具体数字例子】先完整 Forward

取：

$$\mathbf{x} = [2, -1, 3]$$

$$\mathbf{w} = [0.5, -2, 1]$$

$$b = 0.2, \text{target} = 5$$

逐项乘：

项	计算	贡献
$x_1 * w_1$	2×0.5	1.0
$x_2 * w_2$	$(-1) \times (-2)$	2.0
$x_3 * w_3$	3×1	3.0
Bias	+0.2	0.2

所以：

$$z = 1 + 2 + 3 + 0.2 = 6.2$$

$$a = \text{ReLU}(6.2) = 6.2$$

$$\text{error} = a - \text{target} = 1.2$$

$$L = 1.2^2 = 1.44$$

数学直觉：先别急着求梯度

Backward 永远建立在当前 Forward 的具体数值上。本例 ReLU 输入 $z=6.2>0$ ，所以这一次 ReLU 的局部导数是1。

四、【Shape】一个神经元里哪些量是向量，哪些是标量？

变量	Shape	含义
x	(3,)	3 个输入特征
w	(3,)	每个输入对应一个 Weight
b	scalar	整个神经元一个 Bias
z	scalar	加权求和结果
a	scalar	激活后的神经元输出
target	scalar	本样本目标值
L	scalar	本样本 Loss

Backward 后:

梯度	Shape	必须与谁对应
dL/dw	(3,)	每一个 w_i 一个梯度
dL/db	scalar	Bias 一个梯度
dL/dx	(3,)	每一个 x_i 一个梯度
dL/dz	scalar	中间变量 z 的梯度

核心概念: Shape 直觉

梯度是在问 “Loss 对这个变量的每个元素有多敏感”。因此对一个向量求梯度，结果自然也要给这个向量的每个元素一个对应数字。

五、【Backward 第1段】从 Loss 回到神经元输出 a

Loss:

$$L = (a-y)^2$$

令 $\text{error}=a-y$, 则:

$$dL/d\text{error} = 2*\text{error}$$

当前 $\text{error}=1.2$, 所以:

$$dL/d\text{error} = 2.4$$

又因为 $\text{error}=a-y$:

$$d\text{error}/da = 1$$

所以:

$$dL/da = 2.4 \times 1 = 2.4$$

数学直觉: 这 2.4 是什么?

它表示当前神经元输出 a 如果增加一点, Loss 在局部大约以 2.4 倍速度增加。这个“误差信号”接下来还要继续传过 ReLU。

六、【Backward 第2段】穿过 ReLU 回到 z

本例：

$$a = \text{ReLU}(z), z=6.2>0$$

所以局部导数：

$$da/dz = 1$$

链式法则：

$$dL/dz = (dL/da)(da/dz)$$

$$dL/dz = 2.4 \times 1 = 2.4$$

现在我们已经得到最关键的“上游梯度”：

$$g_z = dL/dz = 2.4$$

核心概念：为什么它重要

z 是所有 Weight、Bias、Input 汇合之后形成的标量。只要知道 dL/dz ，再知道 z 对每个输入变量的局部导数，就能把梯度分发回去。

七、【Backward 第3段】z 对每个 Weight 的局部导数

回到:

$$z = x1*w1 + x2*w2 + x3*w3 + b$$

求 $w1$ 的偏导时, 暂时把其他变量固定:

$$dz/dw1 = x1$$

同理:

$$dz/dw2 = x2$$

$$dz/dw3 = x3$$

所以:

$$dL/dwi = (dL/dz) * xi$$

带入数字:

Weight	输入 x_i	$dL/dwi = 2.4 \times x_i$
w1	2	4.8
w2	-1	-2.4
w3	3	7.2

$$dL/dw = [4.8, -2.4, 7.2]$$

数学直觉: 底层直觉

同一个神经元只有一个 dL/dz , 但不同 Weight 前面乘着不同的输入 x_i , 所以每个 Weight 收到的梯度不同。输入越大, 当前样本对对应 Weight 的梯度贡献通常也会按比例放大。

八、【Bias 梯度】为什么 dL/db 就等于 dL/dz ?

因为:

$$z = \dots + b$$

Bias 在 z 里就是直接加上去:

$$dz/db = 1$$

所以:

$$dL/db = (dL/dz)(dz/db) = 2.4 \times 1 = 2.4$$

核心概念: Bias 的特殊之处

Weight 要乘输入, 所以每个 Weight 梯度带一个 x_i ; Bias 不乘任何输入, 只是直接平移 z , 因此它的局部导数是1。

九、【Input 梯度】为什么还要算 dL/dx ?

虽然普通监督训练通常不更新输入 x ，但为了把梯度继续传给更早的网络层，必须知道：

$$dL/dx$$

对 x_1 ：

$$dz/dx_1 = w_1$$

因此：

$$dL/dx_i = (dL/dz) * w_i$$

带入数字：

Input	对应 Weight w_i	$dL/dx_i = 2.4 \times w_i$
x_1	0.5	1.2
x_2	-2	-4.8
x_3	1	2.4

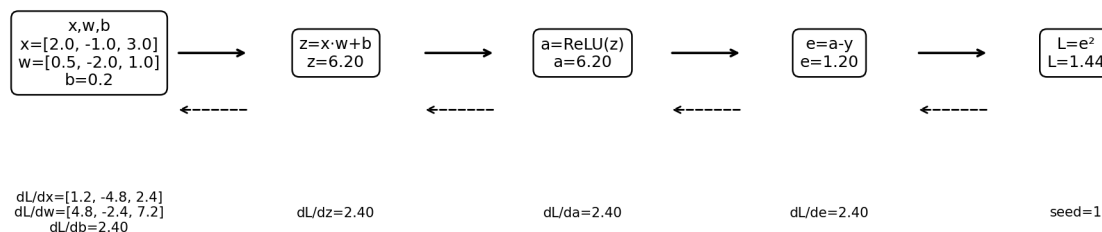
$$dL/dx = [1.2, -4.8, 2.4]$$

科研连接：为什么深层网络必须算输入梯度

对当前层来说 x 是“输入”；但在深层网络里， x 往往是前一层的输出。 dL/dx 就是继续传给前一层的上游梯度。没有它，梯度无法继续向更早期传播。

十、【一张图看完整 Forward / Backward】

单神经元: Forward 算数值, Backward 反向传梯度



你可以把整个反向传播压缩成三条核心公式:

$$dL/dz = dL/da \times da/dz$$

$$dL/dw = (dL/dz) \times x$$

$$dL/dx = (dL/dz) \times w$$

$$dL/db = dL/dz$$

核心概念: 注意方向

Forward 是 $x, w, b \rightarrow z \rightarrow a \rightarrow L$; Backward 是 $L \rightarrow a \rightarrow z \rightarrow w, b, x$ 。Backward 并不是重新“猜参数”, 只是计算当前 Loss 对各变量的局部敏感度。

十一、【向量形式】为什么公式可以写得这么短？

标量逐项公式：

$$dL/dw_i = g_z * x_i$$

把三个 Weight 梯度并在一起：

$$dL/dw = g_z * x$$

同理：

$$dL/dx = g_z * w$$

其中 g_z 是一个标量，所以它相当于把整个向量按同一个比例缩放。

对象	公式	Shape
Weight gradient	$g_z * x$	(3,)
Input gradient	$g_z * w$	(3,)
Bias gradient	g_z	scalar

科研连接：下一讲为什么会出现矩阵乘法

一个神经元只有一个 g_z ；多个神经元、多样本时，许多这样的标量梯度会组织成向量/矩阵，矩阵乘法就成为批量计算这些梯度的工具。

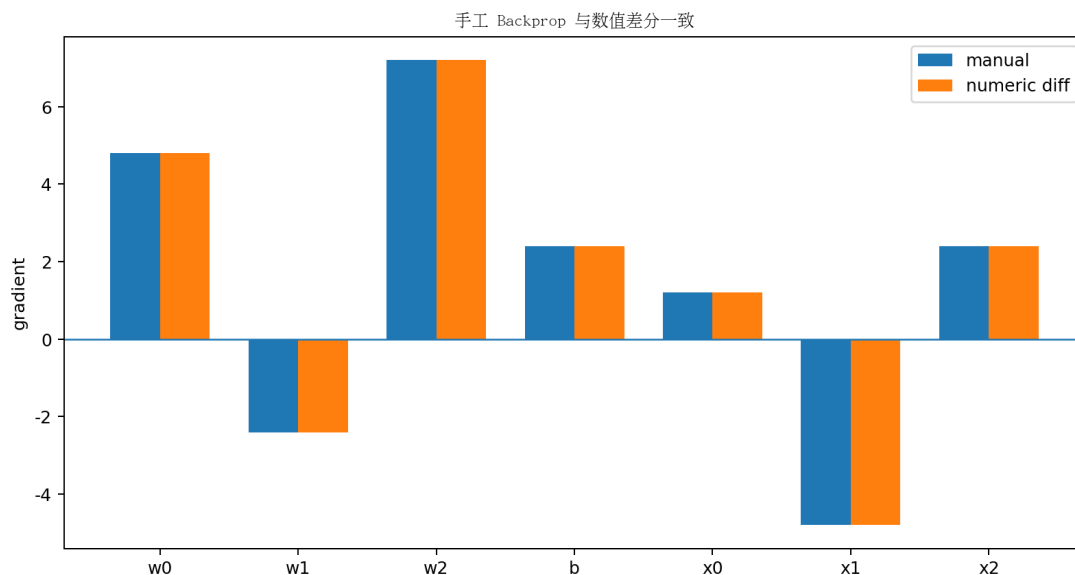
十二、【Gradient Check】怎么知道手工梯度没算错？

对每一个 Weight、Bias、Input 都做 central difference:

$$dL/dq \approx [L(q+eps)-L(q-eps)] / (2eps)$$

实验结果:

变量	手工 Backprop	数值差分	误差量级
w1	4.8	4.8	约2.7e-11
w2	-2.4	-2.4	约8.6e-11
w3	7.2	7.2	约4.1e-11
b	2.4	2.4	约1.4e-11
x1	1.2	1.2	约4.3e-11
x2	-4.8	-4.8	约2.7e-11
x3	2.4	2.4	约8.6e-11



科研连接: Gradient Check 不是训练方法

数值差分需要反复重跑 Forward, 参数多时极慢。它的价值是做正确性检查, 尤其适合你以后自己实现新模块、新 Loss 或手写 backward 时使用。

十三、【一次参数更新】梯度怎样真正影响模型？

取学习率：

$$lr = 0.02$$

最简单的 Gradient Descent：

$$w_new = w - lr * dL/dw$$

$$b_new = b - lr * dL/db$$

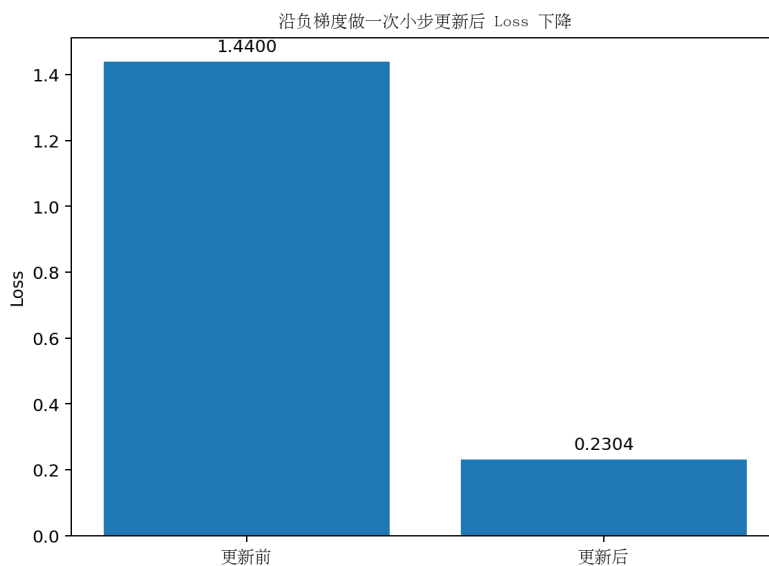
本例更新后：

$$w_new = [0.404, -1.952, 0.856]$$

$$b_new = 0.152$$

重新 Forward：

$$\text{Loss: } 1.44 \rightarrow 0.2304$$



核心概念：这还不是完整训练循环

我们这里只做“一步更新”来说明梯度怎么被使用。真正的 SGD、Momentum、Adam、AdamW 和 Learning Rate 会在第 33 讲以后系统学习。

十四、【ReLU Gate】为什么 Loss 很大，梯度也可能全是0？

换一组 Weight, 让:

$$z = -6$$

$$a = \text{ReLU}(z)=0$$

如果 target=2:

$$L=(0-2)^2=4$$

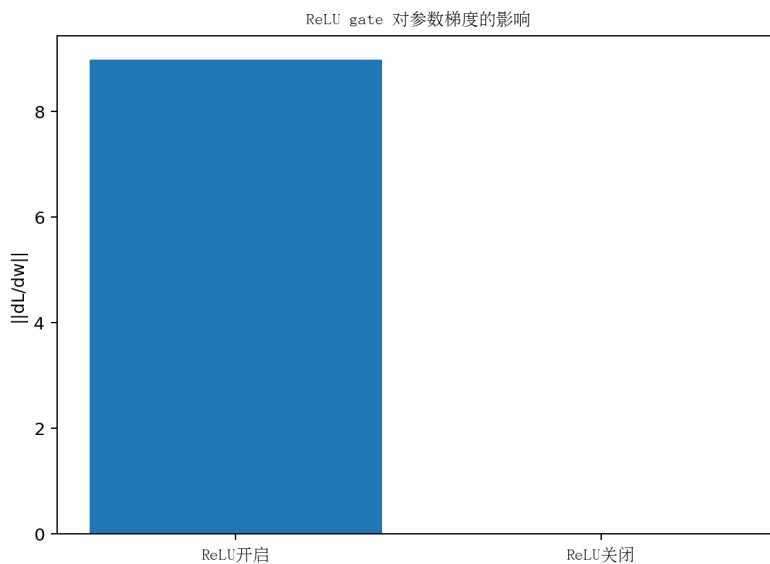
Loss 不小, 但:

$$da/dz = 0$$

于是:

$$dL/dz = 0$$

$$dL/dw = 0, dL/db = 0$$



常见误区 / 风险: 关键结论

Loss 的大小和某个参数 Gradient 的大小不是同一回事。Gradient 必须沿计算图经过所有局部导数, 任何一个局部导数为0, 都可能截断这条路径。

十五、【参数梯度 vs 输入梯度】必须区分

梯度	数学含义	训练中通常做什么
dL/dw	Loss 对 Weight 的敏感度	交给 optimizer 更新 Weight
dL/db	Loss 对 Bias 的敏感度	交给 optimizer 更新 Bias
dL/dx	Loss 对输入 x 的敏感度	继续传给前一层；原始输入通常不更新
dL/dz	Loss 对中间量 z 的敏感度	作为局部 backward 的上游梯度

核心概念：为什么 dL/dx 在深度学习里极其重要

一层的输入 x 往往是上一层的输出。当前层算出 dL/dx 后，就等于把“误差信号”交给上一层，上一层再继续按自己的局部规则往前传。

十六、【Batch 预览】共享 Weight 的梯度怎样聚合？

如果两个样本共享同一套 w, b ，那么每个样本都会对 Weight 产生一份梯度贡献。若 Loss 使用 mean reduction，这些贡献会按平均 Loss 的规则聚合。

量	本讲实验值
两个样本 z	$[6.2, -3.3]$
逐样本 Loss	$[1.44, 1.00]$
Mean Loss	1.22
逐样本 dL/dz	$[1.2, 0.0]$
聚合 dL/dw	$[2.4, -1.2, 3.6]$
聚合 dL/db	1.2

数学直觉：为什么第二个样本贡献为0

第二个样本 $z=-3.3$ ，ReLU 关闭，所以它的 dL/dz 为0。这也是 Batch 中不同样本对同一个参数可以贡献完全不同梯度的原因。

科研连接：这里只是预览

第25讲会正式进入两层 MLP Backpropagation，届时梯度会变成向量和矩阵，矩阵乘法会再次成为主角。

十七、【PyTorch 对照】loss.backward() 最终得到了什么？

本讲实验用 PyTorch 只做验证，不要求现在背语法。对相同 x 、 w 、 b 、 $target$ ：

梯度	手工	PyTorch Autograd
dL/dw	$[4.8, -2.4, 7.2]$	$[4.8, -2.4, 7.2]$
dL/db	2.4	2.4
dL/dx	$[1.2, -4.8, 2.4]$	$[1.2, -4.8, 2.4]$

核心代码：

```
z = (x * w).sum() + b
a = torch.relu(z)
loss = (a - target) ** 2
loss.backward()
```

```
print(w.grad)
print(b.grad)
print(x.grad)
```

核心概念：现在应该怎样理解 `loss.backward()`

不是“PyTorch 会魔法求导”，而是它沿记录好的计算图逆序执行你已经学过的节点级 backward，最终把每个可求导 Tensor 的梯度累积起来。

十八、【与 Transformer / LLM 的关系】

Transformer 里虽然很少画“单神经元”，但底层仍由大量线性映射和非线性组成。

LLM 组件	与本讲的关系
$Q = XW_Q$	每个 W_Q 元素最终都要得到 dL/dW_Q ；本讲是单神经元版本
FFN / MLP	Linear $\rightarrow$ GELU/SiLU $\rightarrow$ Linear，反传仍是局部梯度逐层相乘/累加
Embedding	Embedding 参数只对被访问 token 行产生相应梯度贡献
Cross Entropy	产生最终 loss signal，随后一路反传到所有参与当前预测的参数
Pretraining	本质就是对海量 token 重复 Forward $\rightarrow$ Loss $\rightarrow$ Backward $\rightarrow$ Update

科研连接：你以前在 LLM 课程见过的 Backpropagation

以前你知道“大模型通过反向传播更新参数”；现在你开始知道一个 Weight 的梯度究竟怎样从最终 Loss 经过局部导数一步步计算回来。

十九、【常见误区】

常见误区 / 风险：误区1: Weight 越大, Weight gradient 就一定越大

不一定。 $dL/dw_i = (dL/dz) \times x_i$, 当前梯度取决于上游梯度和输入, 不仅取决于 w_i 自己的数值。

常见误区 / 风险：误区2: dL/dx 没用, 因为输入不训练

错误。深层网络里当前层的 x 往往来自前一层, dL/dx 正是把梯度继续传回前一层的接口。

常见误区 / 风险：误区3: Bias 没有输入, 所以没有梯度

错误。 z 对 b 的局部导数是1, 因此 $dL/db = dL/dz$ 。

常见误区 / 风险：误区4: Loss 大, 所有 Weight 都应该收到大梯度

错误。ReLU gate、饱和激活、长链式乘积等都可能让某些路径梯度很小或为0。

常见误区 / 风险：误区5: Gradient 是“参数应该改成多少”

Gradient 只是局部敏感度/方向信息; 真正更新多少还由 optimizer 和 learning rate 决定。

常见误区 / 风险：误区6: Gradient Check 可以替代 Backprop

不能。Gradient Check 很慢, 只适合小规模验证正确性。

二十、【代码实验】本讲做什么？

实验	问题	结论
1. 完整手工反传	一个 3 输入神经元怎样得到所有梯度	$dw=[4.8,-2.4,7.2]$, $db=2.4$, $dx=[1.2,-4.8,2.4]$
2. Gradient Check	每个元素梯度是否正确	误差约 $1e-10$
3. Weight 梯度来源	为什么不同 w_i 梯度不同	同一 dL/dz 被不同 x_i 缩放
4. 一步更新	梯度怎样被用来降 Loss	$1.44 \rightarrow 0.2304$
5. ReLU 关闭	高 Loss 是否仍可零梯度	可以, 路径被 gate 截断
6. PyTorch 对照	Autograd 是否一致	完全一致
7. Batch 预览	共享参数梯度怎样聚合	样本贡献按 reduction 聚合

二十一、本讲知识闭环



你现在已经能从一个真实神经元的 Loss 出发，手工计算每一个 Weight、Bias、Input 的梯度。

核心概念：下一讲

第25讲：两层 MLP 的 Backpropagation——把今天的单神经元规则扩展到 Linear $\rightarrow$ ReLU $\rightarrow$ Linear，第一次真正看到“多层网络怎样层层传梯度”。